

RAG Knowledge Base — Build Plan & Architecture Report

Pole Star Global · Claude Code Track · 5-Day Assignment

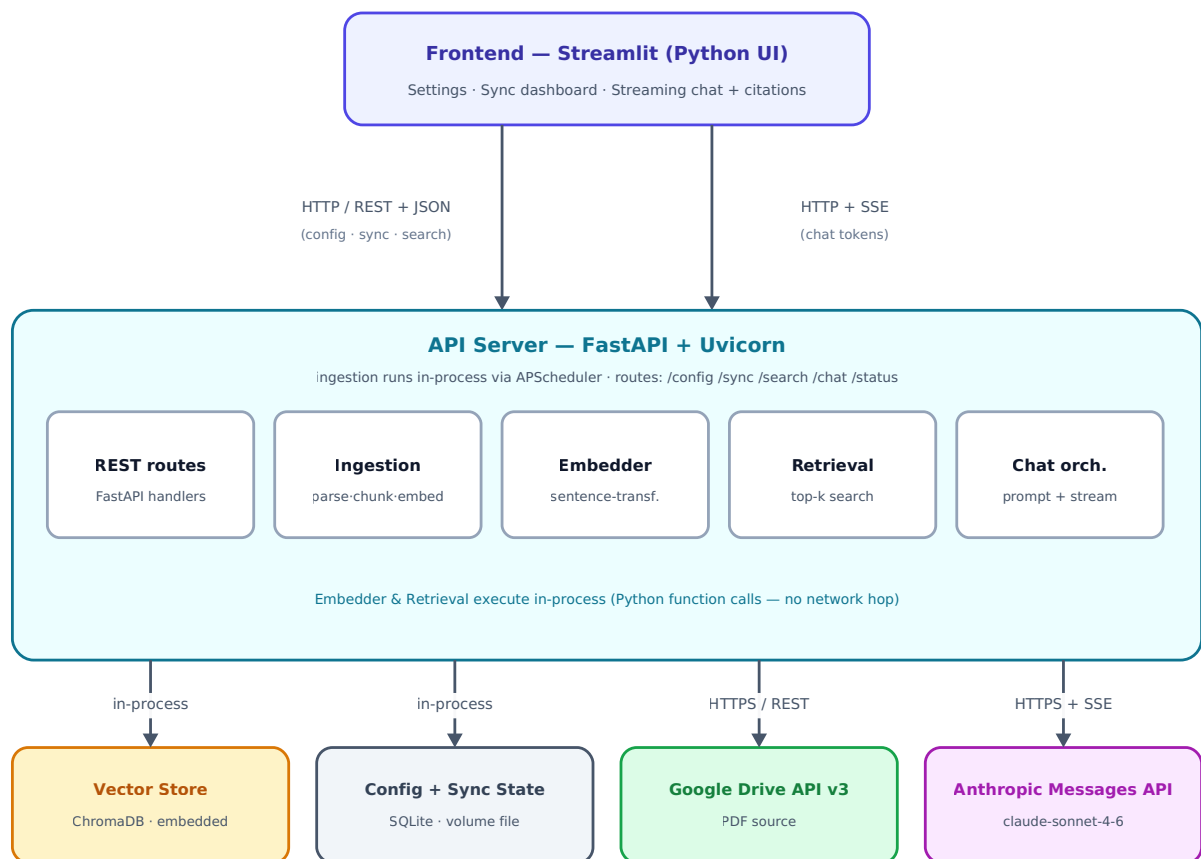
End-to-end build plan · architecture, tech stack, payloads, protocols & edge-case handling · Short version

A short, plain-English plan for the assignment: what to build, in what order, the tech choices and why, the data that moves between the parts, and how tricky cases (like edited or deleted files) are handled. Everything here stays inside the brief's fixed rules, and the choices are made with the **5-day deadline** in mind — simple and fast beats fancy.

The idea in one line: users drop PDFs into a Google Drive folder; the app reads them, splits them into small pieces, turns each piece into a vector, and stores it. When you ask a question, it finds the most relevant pieces and asks Claude to answer using only those — with citations.

A few words first: an *embedding* is a list of numbers that captures a piece of text's meaning; a *chunk* is a small slice of a document; *top-k* means "the k closest matches."

1. The system, and how data flows



Architecture & data flow — each inter-service call labelled with its protocol (reuse this diagram in the README).

Five parts, as the brief requires. To keep things simple for a solo build, the **ingestion worker runs inside the API process** as a background job, and **ChromaDB runs embedded inside the API**

(a library that saves to a volume, not a separate server) — the brief allows this. The Streamlit UI is the only other container; SQLite is just a file on disk.

Getting PDFs in (ingestion):

1. You click *Sync* → the API kicks off a background job.
2. It lists every PDF in the Drive folder.
3. It compares that list to what it synced last time, to spot what's new, changed, or deleted.
4. For new or changed files: download → pull out the text page by page → split into overlapping chunks → embed each chunk → save to ChromaDB with its info (file name, page, chunk number).
5. For deleted files: remove their chunks from ChromaDB.
6. The UI shows progress — files done, chunks created, any errors.

Answering a question:

1. You type a question → the API turns it into a vector the same way.
2. ChromaDB returns the top-k closest chunks.
3. If nothing is close enough, it replies "not in the knowledge base" instead of guessing.
4. Otherwise it puts those chunks into a prompt and asks Claude to answer using only them.
5. Claude's answer streams back word by word, with citations (file + page).

2. The tech stack, and why

Part	Choice	Why (in plain terms)
Backend	Python 3.11 + FastAPI	Brief recommends it; async makes streaming and waiting on Drive/Claude easy; free auto docs
Vector DB	ChromaDB (embedded, on a volume)	Simplest open-source option; runs inside the API as a library; saves to a volume
Embeddings	local <code>sentence-transformers</code> (all-MiniLM-L6-v2)	Free, runs in the container, no outside service to depend on
PDF text	PyMuPDF (pdfplumber for tables)	Fastest; handles multi-column pages; easy page numbers
Chunking	sentence-aware, ~800 tokens, ~15% overlap	Overlap stops ideas getting cut off at the edges
Frontend	Streamlit (Python UI)	Whole UI in Python in a few hours; no extra build/CORS; the UI is only 10 of 100 points
Config + state	SQLite (file on a volume)	Zero setup; holds settings and sync history
Scheduling	APScheduler (in-process)	No extra broker or container needed

Part	Choice	Why (in plain terms)
Streaming	SSE	One-way server→browser; simpler than WebSocket
Deploy	Docker Compose	One command; volumes keep data across restarts

Four choices that need a sentence of explanation:

- **Embeddings — watch the catch in the brief.** It suggests `claude-haiku-4-5` for embeddings, but Claude models don't make embeddings and Anthropic has no embeddings endpoint — they point you to Voyage AI. So I use a free local model by default, with Voyage as an optional paid, higher-quality choice in settings. (The same model must be used for storing and for searching, or the numbers won't line up.)
- **Embedded ChromaDB (the default).** Running Chroma as a library inside the API (`PersistentClient`) is fewer moving parts than a separate Chroma container — no network call, just a function call — and the data still lives on a volume so restarts don't wipe it. A separate Chroma container is a "spare time" upgrade; it only adds one labelled HTTP hop to the diagram.
- **Streamlit for the UI (the default).** It builds the whole settings screen, sync status, and streaming chat in Python in a few hours — no second build system, no CORS. The UI is worth only **10 of the 100 points**, while the backend (chat, Drive, parsing, vector store) is **65** — so the time goes there. Streamlit still calls the FastAPI endpoints over HTTP, so the required API-service separation stays. Next.js is the "spare time" option if you want a fancier, fully custom UI.
- **Overlapping chunks** matter because the brief tests questions whose answer crosses a paragraph break — overlap means the whole sentence appears in at least one chunk.

Trade-offs at a glance:

Part	Upside	Downside
FastAPI	Easy async + streaming	Must move CPU work (embedding) off the main loop
ChromaDB (embedded)	Simplest, in-process, persists, filters	Less proven at huge scale than Qdrant
Local embeddings	Free, no dependency	Lower quality than top paid models; slower on CPU
PyMuPDF	Fast, good with layouts	No text from scanned PDFs (would need OCR)
Streamlit	Whole UI fast in Python	Less custom-looking than React/Next.js
SQLite	Zero ops	One writer at a time
APScheduler	No broker	No durable queue or retries if it crashes
SSE	Dead simple	One-way only

If I had more time: Qdrant for the vector DB (more production-grade), a separate Chroma container and a Next.js frontend (cleaner separation / nicer UI), and Celery for ingestion (durable retries).

3. How the parts talk to each other

From → To	Protocol	Note
Streamlit UI → API (settings, sync, search)	HTTP/REST + JSON	normal requests
Streamlit UI → API (chat)	HTTP + SSE	streamed tokens
API → ChromaDB	in-process call	embedded library, no network
API → Google Drive	HTTPS/REST	Drive API v3
API → Anthropic	HTTPS + SSE	streamed answer
API → SQLite	in-process file read	no network
API → embedder	in-process call	model runs in the same process

Inside Docker Compose, the Streamlit UI reaches the API by name (for example `http://api:8000`). SQLite, the embedder, and ChromaDB all run *inside* the API process, so reaching them is just a function call — no network hop. ChromaDB saves to a mounted volume so its data survives restarts.

4. Payloads (what the data looks like)

Drive — list PDFs:

```
q: "'<FOLDER_ID>' in parents and mimeType='application/pdf' and trashed=false"
fields: "files(id, name, md5Checksum, modifiedTime, size)"
```

Each file comes back with `id`, `name`, `md5Checksum`, `modifiedTime`. Then download the bytes with `files.get_media(fileId=...)`.

ChromaDB — save chunks:

```
collection.add(
  ids=["<file_id>_0", "<file_id>_1"],
  embeddings=[...384 floats..., [...]],
  documents=["chunk text 0", "chunk text 1"],
  metadatas=[{"file_id": "...", "file_name": "policy.pdf", "page": 4, "chunk_index": 0}, ...],
)
```

IDs like `file_id_chunkindex` keep re-syncs clean (no duplicates).

ChromaDB — search:

```
collection.query(query_embeddings=[...], n_results=5, where={"file_id": "..."})
# returns documents, metadatas, distances
```

POST /sync → status:

```
{ "added": 3, "modified": 1, "deleted": 1, "unchanged": 7,
  "chunks_created": 148, "errors": [{"file": "scan.pdf", "reason": "no_text"}],
  "last_sync": "2026-06-25T09:14:03Z" }
```

POST /search :

```
// request
{ "query": "refund policy?", "top_k": 5 }
// response
{ "results": [ { "score": 0.83, "file_name": "policy.pdf", "page": 4, "preview": "Refunds within
14 days..." } ] }
```

POST /chat → SSE stream:

```
event: token
data: {"text": "Refunds"}

event: citations
data: [{"id": 1, "file_name": "policy.pdf", "page": 4}]

event: done
data: {}
```

Anthropic request (the grounding part):

```
{ "model": "claude-sonnet-4-6", "stream": true, "max_tokens": 1024,
  "system": "Answer ONLY from the <context>. If it isn't there, say you don't have it. Cite each
claim with [n] = file + page.",
  "messages": [
    { "role": "user", "content": "<context>\n[1] (policy.pdf p.4) Refunds within 14 days...\n</
context>\n\nQuestion: What is the refund policy?" }
  ] }
```

The retrieved chunks go inside the latest user message, tagged [1] , [2] with their file and page. (Settings like the API key and folder ID are saved in SQLite and shown masked, e.g. `sk-ant-...••••` — never committed to Git.)

5. The chat prompt: grounding and citations

Four things the prompt and flow guarantee:

- **Answers only from context.** The system prompt tells Claude to use only the retrieved text. There's also a safety check: if the best match isn't close enough, skip Claude and reply "not in the knowledge base."
- **Citations.** Each chunk is tagged [1] , [2] ... with its file and page; Claude cites those, and the UI shows a side panel linking them to sources.
- **No making things up.** If the answer isn't in the context, it says so rather than inventing one.
- **Multi-turn.** Chat history is kept per session. Each new question re-searches the documents and only sends the recent history plus the new question's chunks to Claude — so it stays coherent without blowing the token budget.

6. Handling edits, deletions, and other edge cases

This is the heart of "incremental sync." A Drive file's ID stays the same when you rename or edit it, but its `md5Checksum` changes when the content changes. So on each sync, compare what's in Drive now against what's saved:

Situation	How it's spotted	What happens
New file	ID not seen before	Process it; save its record
Edited file	ID exists, checksum changed	Delete its old chunks, then re-process from scratch
Deleted file	tracked ID missing from Drive	Delete its chunks from ChromaDB; remove its record
Renamed only	same ID + same checksum	Just update the name; no re-embedding
Unchanged	ID + checksum match	Skip it

Why delete-then-re-add on an edit: if the new version is shorter, a plain overwrite would leave leftover old chunks pointing at deleted text. Wiping its chunks first keeps things clean.

Other cases that are handled:

Edge case	What it does
Wrong Drive credentials	Show a clear error in the UI, not a crash
No relevant chunks found	Honest "not in the knowledge base" reply
Scanned / image-only PDF	No text to extract → flag it in the report (OCR is optional)
Corrupt or locked PDF	Skip that file, log the error, keep going
Claude rate limit / timeout	Retry with backoff; show a friendly error if it still fails
Embedding model changed	Old numbers won't match → warn the user and re-index everything
Restart	Volumes keep the ChromaDB and SQLite data
Very long chat	Trim old history to stay within the token limit

7. Build plan, in phases (mapped to the 5 days)

- **Phase 0 — Setup (Day 1, morning).** Repo layout, Docker Compose skeleton, FastAPI with a `/health` route, `.env.example`, `.gitignore`. *Done when:* `docker compose up` runs and `/health` returns OK.
- **Phase 1 — Google Drive (Day 1).** Service-account login; list the PDFs in a folder; download a file. *Done when:* you can list and download PDFs, with a clear error on bad credentials.
- **Phase 2 — Parse, chunk, embed, sync (Day 2).** Extract text, chunk with overlap, embed, save to embedded ChromaDB; the new/edited/deleted/unchanged comparison; the `/sync`

endpoint plus status. *Done when:* re-syncing skips unchanged files, editing a file re-embeds only that file, and deleting removes its chunks.

- **Phase 3 — Search (Day 2-3).** A `/search` endpoint that returns the top-k chunks with score, file, and page. *Done when:* a question returns clearly relevant chunks.
- **Phase 4 — Chat (Day 3).** Build the grounding prompt, stream Claude's answer over SSE, add citations and the no-context reply, and keep multi-turn history. *Done when:* answers stream with citations and a 2-3 turn chat stays coherent.
- **Phase 5 — Frontend (Day 4).** Streamlit UI: settings screen, sync button with live status, and a streaming chat with citations (use `st.write_stream` to show tokens as they arrive). *Done when:* the whole thing works in the browser.
- **Phase 6 — Polish (Day 5).** Cover the edge cases above, write the tests, finish the README (with the diagram), and record the demo. *Done when:* tests pass and the README is complete.

Use Claude Code throughout — for scaffolding, for fixing errors (paste the full stack trace), and for a "review this file for bugs and missing error handling" pass before each commit. Jot down what helped and what didn't as you go, for the README reflection.

8. Tests (at least 10)

1. Chunking returns the right number of chunks for a known input.
2. Consecutive chunks actually overlap.
3. No chunk goes over the size limit.
4. Page numbers on chunks are correct.
5. A document shorter than one chunk gives exactly one chunk (no crash).
6. Embeddings have the expected length (384).
7. The same text gives the same embedding; a batch returns one vector per input.
8. Searching for a phrase from a known chunk returns that chunk.
9. `top_k` and the file filter both work.
10. The sync comparison returns the right new/edited/deleted/unchanged sets.
11. A no-context question gives the fallback reply and empty citations.
12. The chat endpoint streams tokens, then citations, then done.

9. Project layout and running it

```
rag-knowledge-base/  
├─ docker-compose.yml  
├─ .env.example          # placeholders only  
├─ .gitignore            # .env, *.db, /secrets  
├─ README.md  
├─ api/                  # FastAPI: routes/, ingestion/, embeddings/, vectorstore/, llm/, db/,  
tests/  
└─ ui/                   # Streamlit: settings, dashboard, chat + citations
```

`docker compose up` starts two containers (api, ui). Two volumes (`chroma-data` , `app-data`), both mounted into the `api` container, keep your vectors and config across restarts. The only setup step is filling in `.env` .

10. Bonus points (optional)

- **Re-ranking** the search results with a cross-encoder — the biggest quality boost (+4).
 - **OAuth login** instead of a service account (+5).
 - **Live sync progress bar** over SSE — easy once chat streaming exists (+3).
 - **DOCX support** alongside PDF (+3).
-

For the review: be ready to explain a few choices in your own words — why a local embedding model instead of an API (free, no dependency, and Claude isn't an embedder), why you delete a file's chunks before re-adding them on an edit (avoids leftover stale chunks), why SSE instead of WebSocket (the streaming only goes one way), and why Streamlit + embedded Chroma over a React frontend and a Chroma server (the deadline — the UI is 10 points, the backend is 65, and embedded Chroma is one less container to run; both can be upgraded later).